



```
1 from queue import PriorityQueue
2
3 def best_first_search(graph, start, target, heuristics):
4     visited = set()
5     pq = PriorityQueue()
6
7     # (heuristic_value, current_node, path)
8     pq.put((heuristics[start], start, [start]))
9     visited.add(start)
10
11     while not pq.empty():
12         h, node, path = pq.get()
13
14         # Goal Test
15         if node == target:
16             return path, h
17
18         # Visit neighbors
19         for neighbor in graph.get(node, []):
20             if neighbor not in visited:
21                 visited.add(neighbor)
22                 pq.put((heuristics[neighbor], neighbor, path + [neighbor]))
23
```

main.py



Share

Run

```
23
24     return None, None
25
26
27 # Example Usage
28 if __name__ == "__main__":
29
30     graph = {
31         'A': ['B', 'C'],
32         'B': ['D', 'E'],
33         'C': ['F'],
34         'D': [],
35         'E': [],
36         'F': []
37     }
38
39     heuristics = {
40         'A': 10,
41         'B': 8,
42         'C': 5,
43         'D': 6,
44         'E': 4,
45         'F': 0
```

```
39     heuristics = {
40         'A': 10,
41         'B': 8,
42         'C': 5,
43         'D': 6,
44         'E': 4,
45         'F': 0
46     }
47
48     path, cost = best_first_search(graph, 'A', 'F', heuristics)
49
50     if path:
51         print(f"Path found by Best-First Search: {path}")
52         print(f"Target Heuristic: {cost}")
53     else:
54         print("No path found.")
```

Output

Path found by Best-First Search: ['A', 'C', 'F']

Target Heuristic: 0

=== Code Execution Successful ===